

Quantization Effects on VLM Inference:

Qwen3.5-2B vs. Qwen3.5-2B-AWQ on MME-RealWorld

Benchmark Report

September 22, 2026

Abstract

We benchmark **Qwen3.5-2B** against its 4-bit AWQ-quantized variant (**Qwen3.5-2B-AWQ**) for vision-language inference under vLLM, evaluating both speed/memory and task accuracy on a stratified 10% sample of MME-RealWorld (2,360 items: 284 Reasoning, 2,076 Perception). Contrary to the throughput and memory gains AWQ quantization typically provides, this run shows AWQ *slower* on wall-clock and per-request latency, with peak GPU memory statistically unchanged, and a **4.6-point (10% relative) drop in overall accuracy**, concentrated in perception-heavy subtasks (OCR, diagrams/tables). We report full latency, throughput, and per-subtask accuracy metrics, and discuss the run’s methodological caveats.

1 Setup and Methodology

Both models were served with **vLLM** on a single NVIDIA **A100 (40 GB)**, back to back in the same process lineage, with the run’s own per-request instrumentation (`RequestOutput.metrics`) driving every timing figure below.

- **Models:** Qwen/Qwen3.5-2B (fp16 baseline) vs. QuantTrio/Qwen3.5-2B-AWQ (4-bit AWQ).
- **Engine config:** `dtype=float16`, `gpu_memory_utilization=0.85` (pinned identically across both models so neither claims extra KV-cache headroom), `max_model_len=32768`, `max_num_seqs=64`; prefix caching *disabled* (unsupported for this Mamba-hybrid architecture in the installed vLLM build).
- **Sampling:** greedy decoding (`temperature=0.0`), `max_tokens=512`.
- **Dataset:** MME-RealWorld, stratified 10% sample (fixed seed) of the Reasoning and Perception splits — $284 + 2,076 = 2,360$ multiple-choice VQA items, identical for both models. Scored with MME-RealWorld’s own `eval_your_results.py`.

2 Performance

2.1 Run-level statistics

† Preemption counts came back null for both runs; the installed vLLM build did not populate `num_preemptions` on `RequestStateStats`. Scheduler preemption cannot be ruled out as a contributor to tail latency on either model.

2.2 Latency decomposition

Per-request timing is broken into queue time (waiting to be scheduled), prefill time (scheduled $\rightarrow$ first token), and decode time (first $\rightarrow$ last token); *latency* is their sum and *TTFT*/*TPOT* are

Metric	fp16	AWQ
Requests	2,360	2,360
Model load time (s)	223.3	274.7
Wall-clock time (s)	1643.7	1775.9
Throughput (req/s)	1.436	1.329
Throughput (tok/s)	155.5	156.4
Corrupted requests	0	0
Truncated requests (hit 512-tok cap)	185 (7.8%)	171 (7.2%)
Preemptions, total	n/a [†]	n/a [†]
Preemptions, requests affected	n/a [†]	n/a [†]
Peak GPU memory (GB)	42.37	42.51
Unload residual memory (GB)	0.449	0.000

Table 1: Aggregate run statistics, 2,360 requests per model.

the standard time-to-first-token and time-per-output-token figures. All values in seconds unless noted.

	avg	p50	p90	p99
<i>End-to-end latency (s)</i>				
fp16	16.583	12.392	37.499	51.229
AWQ	19.464	17.172	39.869	54.167
(token-weighted avg)	fp16: 27.102	AWQ: 26.521		
<i>Time to first token (s)</i>				
fp16	5.260	3.843	12.109	18.751
AWQ	5.770	4.248	13.018	19.778
<i>Time per output token (s)</i>				
fp16	0.434	0.305	1.150	2.734
AWQ	0.408	0.233	0.941	2.502
<i>Queue time (s)</i>				
fp16	1.053	0.269	2.838	6.872
AWQ	1.215	0.838	3.082	8.580
<i>Prefill time (s)</i>				
fp16	4.207	2.597	10.724	18.347
AWQ	4.555	2.800	11.293	18.563
<i>Decode time (s)</i>				
fp16	11.323	3.360	33.511	49.428
AWQ	13.694	8.754	36.175	51.474

Table 2: Latency-family metrics, per model, with standard percentiles. Token-weighted average latency is reported separately where applicable.

Observation. AWQ is slower than fp16 at every percentile of end-to-end latency, TTFT, queue time, prefill time, and decode time, and its wall-clock time and load time are both higher (Table 1). The one metric where AWQ edges ahead is per-token decode speed (TPOT avg 0.408 s vs. 0.434 s), but this does not translate into better aggregate throughput (1.329 req/s vs. 1.436 req/s). This is the opposite of AWQ’s usual value proposition and is discussed further in Section 4.

Task	n	fp16	AWQ	Δ	E-fp16	E-AWQ
Reasoning	284	0.3204	0.3204	+0.0000	56	54
Perception	2076	0.4740	0.4220	-0.0520	249	310
Overall	2360	0.4555	0.4097	-0.0458	305	364

Table 3: Task-level accuracy. $\Delta = \text{AWQ} - \text{fp16}$. *E-fp16/E-AWQ* are counts of “E choice” items — answers the eval script could not parse a clean letter out of. Full per-subtask breakdown in Appendix A.

3 Accuracy

Quantization costs **4.6 accuracy points overall** ($0.4555 \rightarrow 0.4097$, -10% relative), concentrated almost entirely in **Perception** (-5.2 points, -11% relative); **Reasoning** accuracy is unchanged at the task level (0.3204 for both), though the subtask mix underneath shifts in both directions — unsurprising given subtask sample sizes of 8–34 items.

Within Perception, the largest drops are in **OCR with Complex Context** ($0.652 \rightarrow 0.504$, -14.8 points) and **Diagram and Table** ($0.418 \rightarrow 0.356$, -6.2 points), tasks that lean on fine-grained visual/text detail — consistent with quantization degrading the vision-language pathway’s precision more than its general reasoning. **Remote Sensing** also drops ($0.505 \rightarrow 0.457$), driven by the *Count* subtask ($0.261 \rightarrow 0.160$).

Notably, AWQ produced *more* unparseable (“E choice”) answers overall (364 vs. 305) despite *fewer* truncated (max-token-cut-off) requests (171 vs. 185, Table 1) — so the extra parse failures are not simply explained by more truncation; AWQ appears to generate more answers the eval script cannot cleanly extract a letter from, independent of length.

4 Discussion: Anomalies and Limitations

- **The slowdown is unexplained by GPU generation.** On Turing-class GPUs, vLLM falls back to a slower AWQ kernel because the fast Marlin kernel requires Ampere or newer; this run was confirmed to be on an **A100 (Ampere, 40 GB)**, where Marlin *should* be available. The captured engine startup log (fp16 run) does not include an AWQ-specific kernel-selection line, and no equivalent log was available for the AWQ run itself, so which kernel AWQ actually used remains **unconfirmed**. The observed slowdown should therefore be treated as an open anomaly rather than attributed to a kernel fallback.
- **Peak memory is near the device ceiling.** Both models report ≈ 42.4 – 42.5 GB peak allocated on a 40 GB A100. Read literally this exceeds device capacity, which is not physically possible; the more likely explanation is NVML reporting including driver/reserved overhead beyond the CUDA-visible 40 GB, or a measurement quirk in the polling harness. Regardless of the precise cause, the reported figure indicates the run had **little to no memory headroom left**, which is worth confirming before scaling `max_num_seqs` or context length on this hardware.
- **Memory parity is a setup artifact, not a quantization finding.** `gpu_memory_utilization` was deliberately pinned to 0.85 for both models so neither could claim extra KV-cache headroom for a fairer throughput comparison — but a side effect is that vLLM pre-allocates its KV-cache pool up to that fixed ceiling at startup, so peak memory mostly reflects the configured utilization target rather than AWQ’s actual weight-memory savings. A measurement isolating weight footprint alone would need a different protocol (e.g. unpinned utilization, or direct weight-only measurement).

- **Preemption instrumentation did not populate.** `num_preemptions` came back null for every request on both models (Table 1, †), so scheduler preemption cannot be ruled out as a contributor to the long tail latencies observed (p99 $\approx$ 51–54s on both models).
- **7–8% of requests were truncated** at the 512-token generation cap. Truncated requests’ latency and TPOT reflect an artificial cutoff rather than a natural stop, and a cut-off answer may lack a parseable multiple-choice letter — a likely contributor to the “E choice” counts in Table 2.
- **Small per-subtask samples.** Several MME-RealWorld subtasks in this 10% sample have only 8–25 items (e.g. Reasoning/Intention: 8, Reasoning/Property: 10), so subtask-level deltas of a few points are within sampling noise; only the task-level and overall figures (284 / 2,076 / 2,360 items) are reasonably stable.

5 Conclusion

On this run, AWQ quantization of Qwen3.5-2B was a net loss on both axes that typically motivate it: slower end-to-end serving (higher latency at every percentile, lower request throughput, longer load time) and a 10% relative drop in MME-RealWorld accuracy concentrated in perception-heavy subtasks. The only favorable signal is a marginally better per-token decode rate, which did not offset the rest.

The accuracy cost stands independent of any hardware or kernel explanation and should be weighed directly against whatever memory or throughput benefit AWQ is expected to deliver in a target deployment. The speed result, however, should not yet be read as an inherent property of AWQ on this model: confirming the actual AWQ kernel selected at runtime (Marlin vs. fallback) and re-measuring memory with an unpinned `gpu_memory_utilization` would substantially narrow the open questions in Section 4 and should precede any production decision based on these speed numbers.

A Full Per-Subtask Accuracy Breakdown

Table 4: Reasoning task — full subtask and item breakdown. Bold rows are subtask aggregates; indented rows are leaf items.

Subtask / item	n	fp16	AWQ	Δ
Monitoring	46	0.2609	0.3696	+0.1087
Calculate	28	0.3214	0.3929	+0.0715
Property	10	0.3000	0.5000	+0.2000
Intention	8	0.0000	0.1250	+0.1250
OCR with Complex Context	51	0.3922	0.2941	-0.0981
Scene understanding	25	0.2800	0.2400	-0.0400
Character identification	26	0.5000	0.3462	-0.1538
Diagram and Table	42	0.3095	0.3095	+0.0000
Table	30	0.3000	0.2667	-0.0333
Diagram	12	0.3333	0.4167	+0.0834
Autonomous Driving	145	0.3172	0.3172	+0.0000
Relation_interaction_other2other	23	0.2174	0.2174	+0.0000
Relation_interaction_ego2traffic signal	13	0.3077	0.2308	-0.0769
Prediction_intention_vehicle	25	0.4000	0.4000	+0.0000
Relation_interaction_ego2vehicle	12	0.1667	0.4167	+0.2500
Attention_traffic signal	18	0.6667	0.6111	-0.0556
Prediction_intention_ego	34	0.2941	0.2353	-0.0588
Relation_interaction_ego2pedestrian	8	0.3750	0.2500	-0.1250
Prediction_intention_pedestrian	12	0.0000	0.1667	+0.1667

Table 5: Perception task — full subtask and item breakdown. Bold rows are subtask aggregates; indented rows are leaf items.

Subtask / item	n	fp16	AWQ	Δ
Monitoring	221	0.2805	0.3303	+0.0498
Vehicle/attribute	46	0.2391	0.3478	+0.1087
Person/counting	96	0.2292	0.2604	+0.0312
Vehicle/counting	60	0.4167	0.4667	+0.0500
Vehicle/location	10	0.4000	0.4000	+0.0000
Person/attribute	9	0.0000	0.0000	+0.0000
OCR with Complex Context	554	0.6516	0.5036	-0.1480
License	80	0.7250	0.5875	-0.1375
Book_map_poster	149	0.5302	0.3826	-0.1476
Adver_and_product	142	0.6831	0.5493	-0.1338
Text_recog	122	0.6885	0.5164	-0.1721
Phone_and_address	61	0.7049	0.5574	-0.1475
Diagram and Table	536	0.4179	0.3563	-0.0616
Diagram	154	0.4675	0.3831	-0.0844
Table	382	0.3979	0.3455	-0.0524
Autonomous Driving	391	0.3785	0.4143	+0.0358
Objects_identify	103	0.6990	0.7087	+0.0097
Object_count	97	0.2577	0.2680	+0.0103
Attribute_motion_multivehicles/attribute	97	0.1753	0.3196	+0.1443
Attribute_motion_vehicle/attribute	14	0.5714	0.5000	-0.0714
Attribute_visual_traffic signal/attribute	23	0.3478	0.4348	+0.0870
Attribute_motion_multipedestrians/attribute	42	0.2619	0.1429	-0.1190
Attribute_motion_pedestrian/attribute	15	0.4667	0.6000	+0.1333
Remote Sensing	374	0.5053	0.4572	-0.0481

Subtask / item	n	fp16	AWQ	Δ
Count	119	0.2605	0.1597	-0.1008
Position	125	0.7200	0.7120	-0.0080
Color	130	0.5231	0.4846	-0.0385